

NAME :S.GAYATHERI

COURSE CODE : CSA1013

REG NO: 192521306

COURSE NAME : SOFTWARE

ENGINEERING

CO3-ASSESSMENT TOOL - 02

Title: AI-Powered Personalized Nutrition and Health Recommendation Platform

1. Problem Overview

1.1 Introduction

The **AI-Powered Personalized Nutrition and Health Recommendation Platform** is a healthcare and wellness application designed to provide personalized nutrition guidance based on an individual's health information, lifestyle, physical activity, medical history, dietary preferences, and fitness goals.

Traditional diet recommendations are often generic and may not satisfy the specific nutritional requirements of every individual. The proposed system uses Artificial Intelligence to analyze user health data and provide personalized meal recommendations, nutrition monitoring, and continuous health insights.

The platform can help users make better dietary decisions and maintain a healthier lifestyle.

1.2 Problem Statement

Generic dietary recommendations fail to consider individual differences such as age, health conditions, physical activity, dietary preferences, and fitness objectives. This can result in inappropriate dietary choices and poor health management.

The proposed platform addresses this problem by collecting user health information, analyzing nutritional requirements using AI, generating personalized meal plans, monitoring dietary intake, and providing continuous health recommendations.

1.3 Implemented Solution

The system follows a modular architecture in which different functionalities are separated into independent services/modules.

The major components are:

1. User Management
2. Health Data Management
3. AI Recommendation Engine
4. Meal Plan Management
5. Nutrition Tracking

6. Health Monitoring

7. Notification Management

The AI component analyzes the user's available health and lifestyle information and produces suitable nutrition recommendations.

1.4 Project Objectives

The major objectives are:

- To collect personal health information.
- To analyze nutritional requirements using AI.
- To generate personalized meal plans.
- To monitor dietary intake.
- To integrate wearable health devices.
- To provide personalized health recommendations.
- To track nutrition progress.
- To encourage healthy lifestyle adoption.

1.5 Key Functionalities

Functionality	Description
User Management	Registration, login, and profile management
Health Data Management	Stores health parameters and user information
AI Recommendation	Analyzes data and generates recommendations
Meal Planning	Provides personalized meal plans
Nutrition Tracking	Records food and nutritional intake
Health Monitoring	Tracks health and fitness information
Wearable Integration	Supports data from health devices
Notifications	Sends reminders and health-related alerts
Progress Tracking	Displays nutrition and health progress

2. Repository Organization

2.1 Repository Structure

A suitable repository structure for the project is:

AI-Nutrition-Platform/

```
|
|— README.md
|— requirements.txt
|— .gitignore
|— LICENSE
|
|— backend/
|   |— app.py
|   |— routes/
|   |— services/
|   |— models/
|   |— database/
|   |— utils/
|
|— ai_engine/
|   |— model.py
|   |— preprocessing.py
|   |— recommendation.py
|   |— training.py
|
|— frontend/
|   |— pages/
|   |— components/
|   |— assets/
|   |— styles/
```

```
|
|
|— tests/
|   |— test_user.py
|   |— test_health.py
|   |— test_recommendation.py
|   └— test_nutrition.py
|
|— docs/
|   |— architecture.md
|   └— user_guide.md
|
└— screenshots/
    |— login.png
    |— dashboard.png
    └— recommendation.png
```

2.2 Folder Organization

The repository separates the application into logical components.

- **backend/** contains server-side application code.
- **ai_engine/** contains AI and recommendation logic.
- **frontend/** contains the user interface.
- **tests/** contains automated test cases.
- **docs/** contains technical and user documentation.
- **screenshots/** contains evidence for testing and demonstration.
- **README.md** provides project information and instructions.
- **requirements.txt** contains required software libraries.

This separation makes the project easier to understand, test, maintain, and extend.

2.3 File Naming Conventions

Descriptive names such as:

recommendation.py

health_service.py

nutrition_tracker.py

test_recommendation.py

are preferred over unclear names such as:

file1.py

new.py

abc.py

test1.py

Consistent naming makes it easier for developers to identify the purpose of each file.

2.4 Maintainability and Collaboration

The repository structure improves maintainability because developers can work on individual modules without unnecessarily modifying other parts of the application.

For example:

- An AI developer can work inside ai_engine/.
- A backend developer can work inside backend/.
- A testing team can work inside tests/.
- A UI developer can work inside frontend/.

This division also reduces conflicts during collaborative development.

3. Code Quality Review

3.1 Readability

The source code should use:

- Meaningful variable names.
- Meaningful function names.
- Proper indentation.
- Consistent formatting.
- Short and understandable functions.
- Comments for complex logic.

Example of readable code:

```
def calculate_bmi(weight, height):  
    bmi = weight / (height ** 2)  
    return round(bmi, 2)
```

The function name clearly explains its purpose.

3.2 Modularity

The application should be divided into independent modules.

For example:

User Module

↓

Health Data Module

↓

AI Recommendation Module

↓

Meal Planning Module

↓

Nutrition Tracking Module

Modularity makes testing and maintenance easier.

3.3 Coding Standards

The project should follow Python coding conventions such as:

- PEP 8 formatting.
- snake_case for variables and functions.
- PascalCase for classes.
- Meaningful constants.
- Proper exception handling.
- Avoidance of unnecessary duplicate code.

3.4 Documentation

Functions should contain documentation where required.

Example:

```
def generate_meal_plan(user_data):
```

"""

Generates a personalized meal plan
based on the user's health information.

"""

Recommendation logic

3.5 Strengths

The major strengths identified are:

1. Modular architecture.
2. Clear separation of AI and application logic.
3. Meaningful file organization.
4. Use of version control.
5. Potential for automated testing.
6. Easy future integration with wearable devices.
7. Scalable AI recommendation component.

3.6 Areas for Improvement

Possible improvements include:

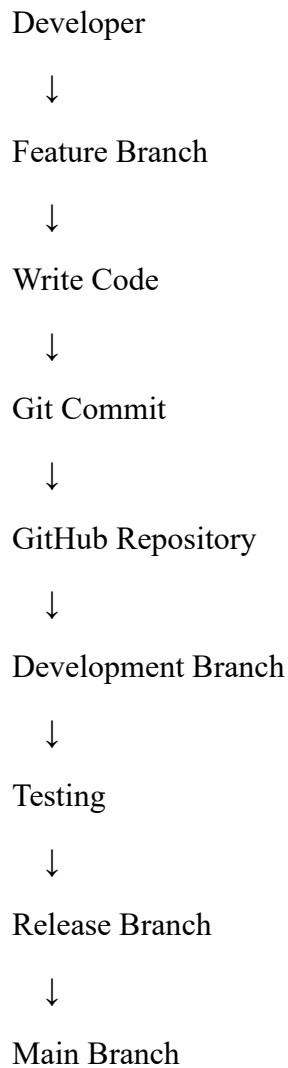
- Add more detailed comments.
- Improve exception handling.
- Add automated unit tests.
- Remove duplicate code.
- Add input validation.
- Improve function documentation.
- Use configuration files for environment-specific settings.
- Avoid storing sensitive credentials in source code.

4. Version Control Evaluation

4.1 Git Repository

Git is used to track changes made to the project source code.

A typical workflow is:



4.2 Commit History

The commit history should contain meaningful messages.

Good examples:

Add user authentication module

Implement BMI calculation

Add AI meal recommendation

Fix nutrition tracking validation

Add recommendation test cases

Update project documentation

Poor examples:

update

changes

final

new

test

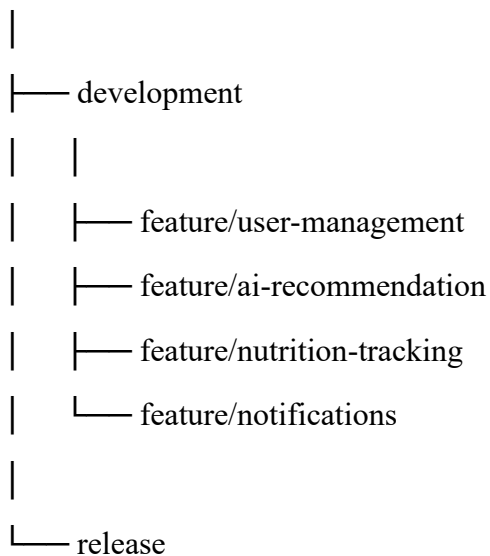
abc

Meaningful commit messages make it easier to understand the development history.

4.3 Branching Strategy

A suitable branching strategy is:

main



The main branch contains stable code.

The development branch contains integrated development work.

Feature branches are used to implement individual functionalities.

4.4 Repository Maintenance

Good repository maintenance practices include:

- Maintaining a .gitignore file.
- Avoiding unnecessary files.
- Writing meaningful commits.
- Keeping the README updated.
- Reviewing pull requests.
- Using branches for new features.
- Removing obsolete branches.

- Avoiding sensitive information in Git.

4.5 Importance of Version Control

Git supports collaborative software development by allowing multiple developers to work on different features simultaneously.

It also provides:

- Change tracking.
- Version history.
- Branch management.
- Collaboration.
- Rollback capability.
- Code review.
- Conflict management.

5. Code Testing and Validation

5.1 Testing Objective

Testing is performed to verify that the application functions according to the specified requirements.

The major modules to test are:

1. User Management
2. Health Data Management
3. BMI/Nutritional Analysis
4. AI Recommendation
5. Meal Planning
6. Nutrition Tracking
7. Notification System

5.2 Test Cases

Test ID	Test Case	Input	Expected Result	Status
TC01	User Registration	Valid user details	Account created	Pass

TC02	User Login	Correct username/password	Login successful	Pass
TC03	Invalid Login	Incorrect password	Error message displayed	Pass
TC04	BMI Calculation	Weight = 60 kg, Height = 1.65 m	BMI calculated	Pass
TC05	Health Data	Valid health parameters	Data stored successfully	Pass
TC06	AI Recommendation	Valid health profile	Personalized recommendation generated	Pass
TC07	Meal Plan	User nutrition profile	Meal plan displayed	Pass
TC08	Nutrition Tracking	Food and quantity	Nutritional intake updated	Pass
TC09	Invalid Input	Missing health data	Validation error displayed	Pass
TC10	Progress Tracking	Previous nutrition records	Progress displayed	Pass

5.3 Example Test

For BMI calculation:

Input:

Weight = 60 kg

Height = 1.65 m

$BMI = Weight / Height^2$

$BMI = 60 / (1.65 \times 1.65)$

$BMI \approx 22.04$

The application should display approximately:

BMI: 22.04

6. Repository Documentation

6.1 README

The README.md file should provide sufficient information for a new developer to understand and execute the project.

It should contain:

1. Project title.
2. Project description.
3. Problem statement.
4. Features.
5. Technologies used.
6. System requirements.
7. Installation instructions.
8. Usage instructions.
9. Testing instructions.
10. Repository structure.
11. Screenshots.
12. Future enhancements.
13. Contributors.

6.2 Installation Instructions

Example:

Step 1: Clone the repository.

Step 2: Open the project directory.

Step 3: Create a virtual environment.

Step 4: Install dependencies.

Step 5: Configure the application.

Step 6: Start the application.

Step 7: Open the application in the browser.

6.3 Dependencies

The required dependencies should be listed in requirements.txt.

Example:

numpy

pandas

scikit-learn

matplotlib

flask

pytest

The exact dependencies should match the libraries actually used by the project.

6.4 Usage Guide

The README explain the basic user workflow:

Login



Enter Health Information



Submit Health Profile



AI Analysis



Personalized Recommendation



Meal Plan



Track Nutrition



View Progress

- Integration testing.
- API testing.
- UI testing.
- Performance testing.
- Security testing.
- AI model validation.
- Edge-case testing.
- Installation guide.
- User guide.

- Developer guide.

Conclusion

The code review of the **AI-Powered Personalized Nutrition and Health Recommendation Platform** demonstrates that the project has significant potential as an AI-based healthcare and wellness solution.

The system addresses the limitations of generic dietary recommendations by using individual health and lifestyle information to generate personalized nutrition guidance. A well-organized Git repository, modular source code, meaningful commit history, comprehensive testing, and proper documentation can significantly improve the project's quality and maintainability.

The recommended improvements in code quality, version control, testing, documentation, security, and future AI capabilities will help transform the project into a more reliable, scalable, and maintainable software solution.